

Ch11_Session and Cookies

What are Cookies?

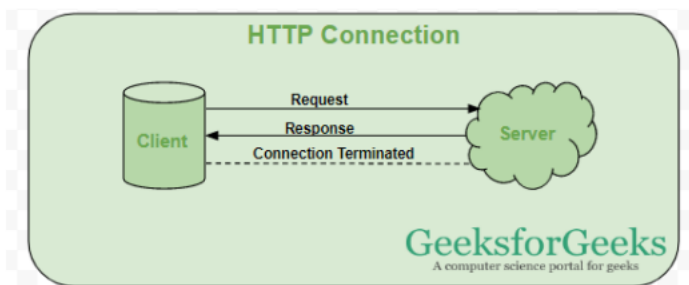
HTTP Cookies

- Small text files of data **created by a web server**

Placed on the user's computer or other device by the user's web browser.

- HTTP => **Stateless protocol**

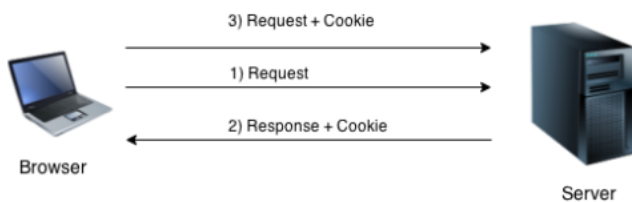
When any request is sent to the server, the server has no idea if the user is new or has visited the site before.



WHY the need for cookies and session?

- All communication between web browsers and servers is via **HTTP**, which is *stateless*
- Cookies and sessions allow you to store arbitrary data per browser to keep track of the "state" between the site and a particular browser.

HOW cookies work?

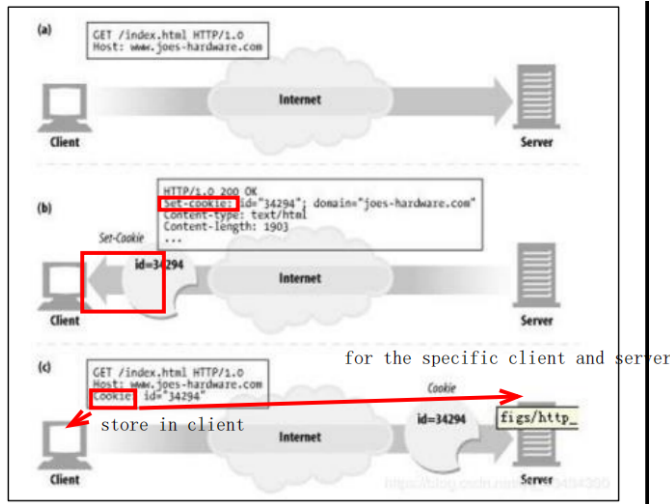


1. **请求 (Request)**: 当用户请求访问某个网站时，浏览器会向服务器发出请求以获取所需页面的内容。
2. **响应 + Cookie (Response + Cookie)**: 服务器返回请求的页面内容，并可能附带一个或多个 **Cookie**。这些 **Cookie** 会被浏览器保存到持久缓存persistent browser cache中。
3. **请求 + Cookie (Request + Cookie)**: 之后，当浏览器再次向同一服务器发出请求时，会**自动附带之前存储的相关 Cookie**。服务器利用这些 **Cookie** 信息来理解请求的上下文并提供更适配的响应。

=> The server can then interpret the cookies as part of the request's context and generate a response to suit.

这个过程会在每次浏览器发出新请求时重复，直到 Cookie 到期或浏览器删除Cookie。=> The browser repeats the process until the **cookie expires** or the **session is closed** and the **cookie is deleted** by the browser itself

eg. the duration of the cookie -> how long you automatically logout



Shortcomings of using Cookies

- **敏感信息的传递风险**：将用户的敏感信息（例如信用卡号）作为 Cookie 的形式在电子商务网站上传递，是非常不明智的设计。这种行为可能导致安全漏洞，给恶意程序拦截 Cookie 创造机会。
- **信息泄露的后果**：一旦 Cookie 被恶意程序窃取，黑客可能会利用其获取用户的敏感信息，比如信用卡号。这表明，如果网络应用的设计存在根本性缺陷，用户的信息安全将面临巨大风险。
- Because of the potentially sensitive nature of cookies, lawmakers have taken a particularly keen interest in them.
- In particular, EU lawmakers in 2011 introduced an EU-wide 'cookie law', where all hosted sites within the EU should present a cookie warning message when a user visits the site for the first time.



Limitations of Cookies

- It is possible to trace users.
- The cookies can be deleted by the client.
- A cookie can only store up to 4096 bytes of information.
- Cookies can be stored by both the browser and the server. session
- Cookies are **plain text files**, and attackers can easily intercept cookies that are not delivered via HTTPS. 拦截

Django Cookies and Cookies Handling

Cookies – 服务器分配给客户端，即服务器通过 HTTP 响应中的 `Set-Cookie` 头将 Cookie 发送到浏览器，浏览器再在随后的请求中携带这些 Cookie。

Workflow for client-side cookies

1.read the cookie files and identify whether or not your host have the cookie from my server
2.if exist -request ->read
if not respond -> create

If client-side cookies are the right approach for you, then work through the following steps:

1. You can retrieve the value of a cookie with `request.COOKIE['cookie_name']`.
 - cookies are returned as strings, regardless of what they contain. You might need to cast to the correct type.
 - However, if the cookie doesn't exist, it will throw an error.
 - To test if a cookie 'cookie_name' exists: `if 'cookie_name' in request.COOKIE`
2. An alternative is to use `request.COOKIE.get('key', 'default')` which returns the value if the key exists, otherwise 'default' - you can put anything you like in place of 'default'.
E.g. `count = int(request.COOKIE.get('cookie_name', '1'))`
3. To update the cookie, pass the value you wish to save to the response you generate.
`response.set_cookie('cookie_name', value)` is the function you call, where two parameters are supplied: the name of the cookie, and the value you wish to set it to.
4. To delete a cookie, use `response.delete_cookie('cookie_name')`

remove the cookie of the client host

上述内容是客户端操作Cookie的流程=>如何通过代码在客户端 读取、检查、更新和删除 Cookies。

- **读取 Cookies**: 通过 `request.COOKIE` 获取 Cookie 的值，用于检查客户端是否已存储某个特定的 Cookie。
- **设置和更新 Cookies**: 通过 `response.set_cookie` 方法在生成的 HTTP 响应中附加新的 Cookie。
- **删除 Cookies**: 通过 `response.delete_cookie` 从客户端删除特定的 Cookie。

用户更新cookie? => Cookies存储在用户浏览器，浏览器可以通过用户端脚本（如JavaScript）来操作这些 Cookies => 某些短期存储的数据可以直接由客户端更新，从而减少对服务器的依赖，降低服务器负载。/ 但是这种能力虽然提升了灵活性，也带来了安全隐患。=> 开发者经常通过设置 **HttpOnly** 和 **Secure** 属性来限制 Cookie 的访问权限，避免被客户端脚本随意更改。

- **Name（名称）**：每个 Cookie 都有一个名称，通常称为 **key**，它是用来标识该 Cookie 的唯一标识符。
- **Value（值）**：与名称相关联的数据。例如，一个 **session_id** Cookie 的值可能是服务器生成的会话标识符。

Session

Sensitive information – save in server

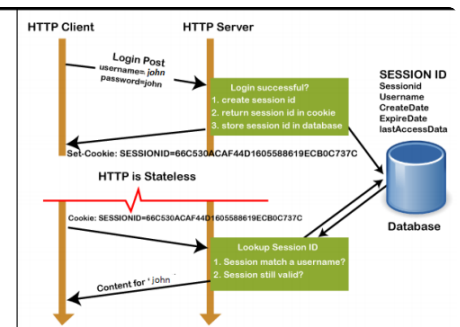
但是 less important things 还是存储在cookie中，因为如果所有东西都存储在了server，对于服务器来说压力很大的。

since **cookies** are stored locally, and those cookies which are not sent over **HTTPS** can be *easily caught by attackers*. => DANGEROUS

Using Django sessions

Using Django sessions

- Django provides a session framework to store arbitrary data for each visitor.
- [Session data is stored on the server side](#), and cookies contain the session ID unless you use the cookie-based session engine.
- The default session engine stores session data in the database, but you can choose between different session engines. When we migrate the application, we can see the [django_session](#) table in the database.
- The session middleware manages the sending and receiving of cookies.
- To use sessions, you have to make sure that the MIDDLEWARE setting of your project contains `'django.contrib.sessions.middleware.SessionMiddleware'`, which is added by default to the MIDDLEWARE setting when you create a new project using the [startproject](#) command.



要使用会话，需确保项目的 **MIDDLEWARE** 设置包含 `'django.contrib.sessions.middleware.SessionMiddleware'`（在使用 **startproject** 命令创建新项目时，默认已添加）。

- Session：数据存储在**服务器端**，而客户端通常只保存一个 `Session ID`（通常通过 Cookie 存储）

Session 是基于 Cookie 的

1. **服务器生成 Session**：当用户首次访问网站时，服务器会创建一个 Session，并分配唯一的 Session ID。
2. **Session ID 存储在 Cookie 中**：服务器通过 HTTP 响应将这个 Session ID 保存到客户端的 Cookie 中。
3. **会话维持**：在后续请求中，浏览器会自动携带该 Session ID（存储在 Cookie 中），服务器根据这个 ID 来识别用户的会话并提供对应的数据。

- **Cookie 适合存储不敏感信息**：例如偏好设置、非关键状态。
 - eg. 用户偏好（例如网站的语言设置 `lang=en`）直接存储在 Cookie 中，无需每次访问都请求服务器。
- **Session 适合处理敏感信息**：例如用户登录状态、账户权限管理。
 - eg. 登录网站后，服务器通过 Session 保存用户的身份信息。客户端只携带 Session ID，服务器根据该 ID 返回登录用户的相关数据。

特性	Session	Cookie
存储位置	服务器端（数据存储）	客户端（浏览器）
敏感数据存储	更安全，数据由服务器管理	较不安全，需要额外保护
生命周期	通常是临时的，随着会话结束清除	可长期存储，设置有效期
数据容量限制	受服务器存储能力限制	每个 Cookie 通常最大为 4KB
适用场景	登录状态、认证信息	用户偏好、非关键状态信息

Cookie 的作用

Cookie 是一种轻量化的客户端存储工具，适合用来传递少量信息，例如：

1. **会话标识符**：Cookie 存储的是服务器分配的 Session ID（例如 `session_id=123abc`），这个 ID 是服务器端会话数据的索引。客户端通过 Cookie 发送这个 Session ID，服务器端会根据 Session ID 查询并管理会话数据。
2. **非敏感数据**：Cookie 可以存储一些简单的用户偏好设置（例如语言或主题颜色），这些信息通常不

需要与服务器交互。

完全基于 Cookie 的会话（即所有会话数据都直接存储在 Cookie 中）

包括敏感信息...

Middleware

Middleware

- Middleware is a framework of hooks into Django's request/response processing.

```
/home/COMP222/django_projects/blog_project/blog_project/settings.py
43
44 - MIDDLEWARE = [
45     'django.middleware.security.SecurityMiddleware',
46     'django.contrib.sessions.middleware.SessionMiddleware',
47     'django.middleware.common.CommonMiddleware',
48     'django.middleware.csrf.CsrfViewMiddleware',
49     'django.contrib.auth.middleware.AuthenticationMiddleware',
50     'django.contrib.messages.middleware.MessageMiddleware',
51     'django.middleware.clickjacking.XFrameOptionsMiddleware',
52 ]
```

Here is the built-in Middleware classes the django-admin startproject command sets up

- Each middleware component is responsible for doing some specific function.
 - For example, the middleware component, [AuthenticationMiddleware](#), associates users with requests using sessions. It adds the user attribute, representing the currently-logged-in user, to every incoming HttpRequest object.
 - **CSRF protection middleware** adds protection against Cross Site Request Forgeries by adding hidden form fields to POST forms and checking requests for the correct value.
- Django's built-in middleware reference – <https://docs.djangoproject.com/en/3.1/ref/middleware/>

Middleware 是 Django 中的一种框架，用于挂钩（hook）到请求和响应的处理过程中。

▼ 代码

SecurityMiddleware：提供安全相关的改进，例如 HSTS（HTTP 严格传输安全）。

SessionMiddleware：启用会话支持，管理与用户的交互状态。

CommonMiddleware：处理一些通用功能，例如 URL 正规化和隐藏不必要的 404 错误。

CsrfViewMiddleware：保护应用免受跨站请求伪造（CSRF）攻击。/ 验证隐藏表单字段中的 CSRF 令牌，确保请求合法性。

AuthenticationMiddleware：通过会话将用户与请求关联起来。

MessageMiddleware：支持消息框架，例如显示通知。

XFrameOptionsMiddleware：添加 HTTP 头，防止点击劫持。

Middleware 是 Django 的一个处理链，它把每个请求和响应包装起来。它可以在以下阶段运行逻辑：

1. **请求处理**：在请求到达视图之前，中间件可以对请求进行预处理。例如，检查用户认证状态或过滤请求头。

2. **响应处理**：在视图返回响应之后，中间件可以修改或增强响应，例如添加安全性标头或处理跨域问题。

请求处理流程

- 1. **用户发起请求**：用户的浏览器或客户端通过 HTTP 协议向服务器发送请求，例如访问网站某个页面或提交表单。
- 2. **Django 接收请求**：服务器（通常是运行 Django 的 Web 服务器，如WSGI）接收这个请求并传递给 Django。
- 3. **Middleware 预处理请求**：
 - 请求在进入具体的视图函数之前，会经过 Django 项目中定义的一系列 Middleware。这些 Middleware 按顺序执行，可以对请求进行预处理，比如：
 - 验证请求的合法性（如 CSRF 验证）。
 - 修改或添加请求的属性（如添加用户认证信息）。
 - 规范化请求路径（如处理尾部斜杠）。
 - 如果 Middleware 发现某些问题（例如无权限访问），可以直接拦截请求并返回响应，而不进入视图函数。
- 4. **请求到达视图函数**：只有在所有 Middleware 执行完毕后，请求才会到达视图函数，视图根据具体的业务逻辑生成响应。

Middleware 在这一阶段的作用

Middleware 的核心作用是“拦截”和“过滤”请求，以确保视图函数只处理合法且经过规范化的请求。例如：

- **身份验证**：检查请求是否包含有效的用户身份凭据。
- **安全性处理**：阻止跨站请求伪造攻击或恶意访问。
- **全局行为设置**：为整个项目的请求统一添加特定的处理逻辑（如安全标头）。

特性	Session	Cookie
存储位置	服务器端	客户端（浏览器）
需要 Middleware 的原因	需要 Middleware 解析和加载服务器端的会话数据	浏览器自动处理，无需服务器端解析

主要作用	提供安全的、动态的会话数据存储	存储轻量、非敏感的客户端状态数据
------	-----------------	------------------

Session 是由**服务器端**管理的机制，因此需要**中间件**来帮助处理会话数据的加载与保存。

而 **Cookie** 是由**浏览器**管理的客户端机制，**无需**服务器端的额外支持即可直接使用。-- **Cookie** 的生命周期、路径、域等属性直接由客户端浏览器管理，服务器只需简单地创建或读取它们。

▼ 为什么Session需要Middleware?

1. Session 是服务器端的机制：

- Session 的核心是通过唯一的 **Session ID** 将客户端与服务器端的会话数据关联起来。
- 数据存储**在服务器端，需要一个机制将客户端发来的 **Session ID** 与服务器端的存储数据（例如数据库中的会话记录）进行匹配。

2. Middleware 的作用：

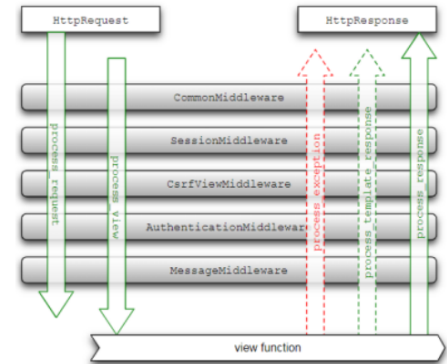
- SessionMiddleware 是 Django 中负责处理会话的中间件。
- 具体作用：**
 - 解析 Cookie 中的 Session ID：**它会从客户端发来的 Cookie 中提取 **Session ID**。
 - 加载会话数据：**根据这个 ID，从服务器端会话存储（如数据库、缓存）中加载对应的数据，并将它挂载到请求对象的 **request.session** 属性上。
 - 保存会话数据：**当视图完成处理后，Middleware 会将任何修改过的会话数据写回服务器端存储。

没有 Middleware 的问题：

- 如果没有 Middleware，Django 无法自动加载和保存服务器端的会话数据。开发者需要手动处理这些过程，这会增加复杂性。

Middleware: How it works

- The Middleware classes are called twice during the request/response life cycle.
- During the **request** cycle, the Middleware classes are executed top-down, meaning it will first execute SecurityMiddleware, then SessionMiddleware all the way until XFrameOptionsMiddleware. For each of the Middlewares it will execute the **process_request()** and **process_view()** methods.
- At this point, Django will do all the work on your view function. After the work is done (e.g. querying the database, paginating results, processing information, etc), it will return a **response** for the client.
- During the **response** cycle, the Middleware classes are executed bottom-up, meaning it will first execute XFrameOptionsMiddleware, then MessageMiddleware all the way until SecurityMiddleware. For each of the Middlewares, it will execute the **process_exception()**, **process_template_response()** and **process_response()** methods.



<https://simpleisbetterthancomplex.com/tutorial/2016/07/18/how-to-create-a-custom-django-middleware.html>

Middleware 的运行过程

1. 请求周期 (Request Cycle) :

- 自上而下执行：当用户的 HTTP 请求进入 Django 时，Middleware 按照定义顺序依次执行。
- 每个 Middleware 会调用其 `process_request()` 方法，预处理请求。例如：
 - 验证用户的身份。
 - 规范化请求路径。
 - 检查请求的合法性（如 CSRF 验证）。
- 在视图函数执行之前，还可能调用 `process_view()` 方法，以便根据视图的具体内容进一步操作。

2. 视图处理阶段：

- 视图函数处理请求，例如查询数据库、分页结果、返回页面内容等。
- 视图完成后生成一个响应对象，进入响应周期。

3. 响应周期 (Response Cycle) :

- 自下而上执行：响应返回时，Middleware 按定义的逆序依次执行。
- 每个 Middleware 会调用其 `process_exception()` 方法处理视图函数可能抛出的异常，或调用 `process_template_response()` 方法对模板响应进行处理。
- 最后，Middleware 调用 `process_response()` 方法，对最终的响应进行修改。例如：
 - 添加安全性 HTTP 标头（如 `X-Frame-Options`）。
 - 设置 Cookies 或自定义响应头。

Session framework

- The session framework lets you store and retrieve arbitrary data on a per-site-visitor basis.
- It stores [data on the server side](#) and abstracts the sending and receiving of cookies.
- [Cookies contain a session ID](#) – not the data itself (unless you're using the cookie based backend).
- To enable session functionality, make sure the MIDDLEWARE setting contains '[django.contrib.sessions.middleware.SessionMiddleware](#)'. The default settings.py created by django-admin startproject has SessionMiddleware activated.
- If you don't want to use sessions, you might remove the SessionMiddleware line from MIDDLEWARE and 'django.contrib.sessions' from your INSTALLED_APPS. It'll save you a small bit of overhead.

Configuring the session engine

- By default, Django stores sessions in your database (using the model `django.contrib.sessions.models.Session`).
- Though this is convenient, in some setups it's faster to store session data elsewhere, so Django can be configured to store session data on your filesystem or in your cache.
- Django offers the following options for storing session data:
 - **Database sessions:** Session data is stored in the database. This is the default session engine.
 - **File-based sessions:** Session data is stored in the filesystem. E.g. set the `SESSION_ENGINE` setting to "[django.contrib.sessions.backends.file](#)".
 - **Cached sessions:** Session data is stored in a cache backend. You can specify cache backends using the CACHES setting. Storing session data in a cache system provides the best performance.
 - **Cached database sessions:** Session data is stored in a write-through cache and database. Reads-only use the database if the data is not already in the cache.
 - **Cookie-based sessions:** Session data is stored in the cookies that are sent to the browser.

会话存储引擎的类型

1. 数据库会话 (Database Sessions):

- 默认引擎：会话数据存储在数据库中，使用 Django 提供的 `django.contrib.sessions.models.Session` 模型。
- 适用场景：适合需要可靠存储和长时间保留会话数据的场景。

2. 基于文件的会话 (File-Based Sessions):

- 将会话数据存储在服务器的文件系统中。
- 配置方法：将 `SESSION_ENGINE` 设置为 `django.contrib.sessions.backends.fil`

e。

- 优点：实现简单，适合小型项目。

3. 基于缓存的会话 (Cached Sessions):

- 会话数据存储在缓存中，例如 Redis 或 Memcached。
- 配置缓存的方式通过 `CACHES` 设置。
- 优点：提供最快的性能，适用于需要高频率访问会话的应用。

4. 缓存 + 数据库会话 (Cached Database Sessions):

- 会话数据采用缓存和数据库结合的方式，缓存负责快速读写，数据库作为回退机制（write-through 模式）。
- 适用场景：需要性能与持久性兼顾的场景。

5. 基于 Cookie 的会话 (Cookie-Based Sessions):

- 将会话数据直接存储在客户端的 Cookie 中。
- 配置方法：将 `SESSION_ENGINE` 设置为 `django.contrib.sessions.backends.signed_cookies`。
- 优点：不需要服务器端存储，适合没有敏感数据的小型项目。
- 注意：需要对 Cookie 数据进行签名和加密，确保数据安全。

Accessing the current session

- You can access the current session using `request.session`, treating it like a Python dictionary to store and retrieve session data.

- You can set a variable in the session like this:

```
request.session['foo'] = 'bar'
```

Note that unlike cookies, setting the value of a session variable is not a response (`response.set_cookie()`).

- Retrieve a session key as follows: `request.session.get('foo')`
- Delete a key you previously stored in the session as follows: `del request.session['foo']`

automatically done by django

15

！我们可以直接通过`request.session`访问和操作会话数据，是因为会话管理自动由中间件完成。虽然开发者无需直接处理底层中间件的逻辑，但它在幕后发挥了重要作用。

SessionMiddleware

- **解析 Cookie**: 从客户端发送的请求中提取 `Session ID` (存储在 Cookie 中)。
- **加载会话数据**: 根据 `Session ID` 从服务器端的存储引擎 (例如数据库、缓存) 中加载对应的会话数据。
- **挂载到请求对象**: 将会话数据加载到 `request.session` 中, 供视图或其他组件使用。
- **保存修改**: 如果会话数据被修改 (例如添加或删除变量), 中间件会在响应阶段将这些更新写回存储引擎。
- 中间件负责所有与会话存储相关的操作, 开发者只需使用 `request.session` 访问数据
- 开发者无需直接管理会话存储引擎或手动处理 Session ID。

▼ 设置会话变量

Python |

```
1 request.session['foo'] = 'bar'
```

通过 `request.session`, 像操作 Python 字典一样存储数据。

注意, 这与设置 Cookie 不同, 不需要通过响应来设置。

▼ 读取会话变量

Python |

```
1 request.session.get('foo')
```

使用 `.get()` 方法获取会话中的数据, 避免变量不存在时抛出异常。

▼ 删除会话变量

Python |

```
1 del request.session['foo']
```

删除已存储的会话变量, 就像从字典中移除键值对一样。

- **SESSION_COOKIE_AGE**: The duration of session cookies in seconds. The default value is 1209600 (two weeks).
- **SESSION_EXPIRE_AT_BROWSER_CLOSE**: A boolean indicating that the session has to expire when the browser is closed.

True/False

By default -> false

- You can choose to use browser-length sessions or persistent sessions using the `SESSION_EXPIRE_AT_BROWSER_CLOSE` setting.
- This is set to False by default, forcing the session duration to the value stored in the `SESSION_COOKIE_AGE` setting.
- If you set `SESSION_EXPIRE_AT_BROWSER_CLOSE` to True, the session will expire when the user closes the browser, and the `SESSION_COOKIE_AGE` setting will not have any effect.

`SESSION_EXPIRE_AT_BROWSER_CLOSE = True:`

- 会话将在浏览器关闭时立即过期，无论 `SESSION_COOKIE_AGE` 设置了什么值。
 - 适合需要较高安全性的场景，例如银行网站或敏感数据的应用。
-

Secure attribute

- To ensure that cookies are sent securely and are not accessed by unintended parties or scripts: the Secure attribute and the HttpOnly attribute.
- A cookie with the `Secure` attribute is sent to the server only with an encrypted request over the HTTPS protocol, never with unsecured HTTP (except on localhost), and therefore can't easily be accessed by a man-in-the-middle attacker.
- However, do not assume that Secure prevents all access to sensitive information in cookies; for example, it can be read and modified by someone with access to the client's hard disk (or JavaScript if the HttpOnly attribute is not set).
- `SESSION_COOKIE_SECURE`: Default is FALSE

21

HttpOnly attribute

- A cookie with the **HttpOnly** attribute is inaccessible to the JavaScript `Document.cookie` API; it is sent only to the server.
- For example, cookies that persist server-side sessions don't need to be available to JavaScript, and should have the HttpOnly attribute. This precaution helps mitigate cross-site scripting ([XSS](#)) attacks.
- **SESSION_COOKIE_HTTPONLY**: Default is TRUE

HttpOnly 属性的功能

1. 不可访问性:

- 启用了 **HttpOnly** 属性的 Cookie 无法通过 JavaScript 的 `document.cookie` API 访问。
- 它仅在客户端与服务器之间的 HTTP 请求中传递，确保 Cookie 数据对浏览器脚本不可见。

2. 应用场景:

- 适用于服务器端会话的持久化 Cookie（例如会话标识符），因为这些数据通常不需要被客户端直接访问。
- 对于此类 Cookie，使用 HttpOnly 属性可以防止未经授权的 JavaScript 代码访问它们。

3. 安全性:

- HttpOnly 是防止 **跨站脚本攻击 (XSS)** 的关键措施之一。即使恶意脚本注入了页面，也无法通过脚本访问带有 HttpOnly 属性的 Cookie。

Django 的默认设置

- **SESSION_COOKIE_HTTPONLY** :
 - 在 Django 中，与会话相关的 Cookie 默认启用了 HttpOnly 属性。
 - 默认值为 **True**，确保会话标识符不会被客户端脚本获取，从而增强会话数据的安全性。